STUDY NOTES · PROJECT DETAILS

What we are building, and why the world wants it

A living file. Each chapter is one big picture and a few lines. New things about the project get added at the end, in the **Project log**.

1 The project in one picture

2 The problem: strangers' code on your computer

3 Three ways to build walls

4 Who uses this in industry

5 Why now, and what they are chasing

6 Our road: Phase 1 → 2 → 3

7 The one rule: the guest is a stranger

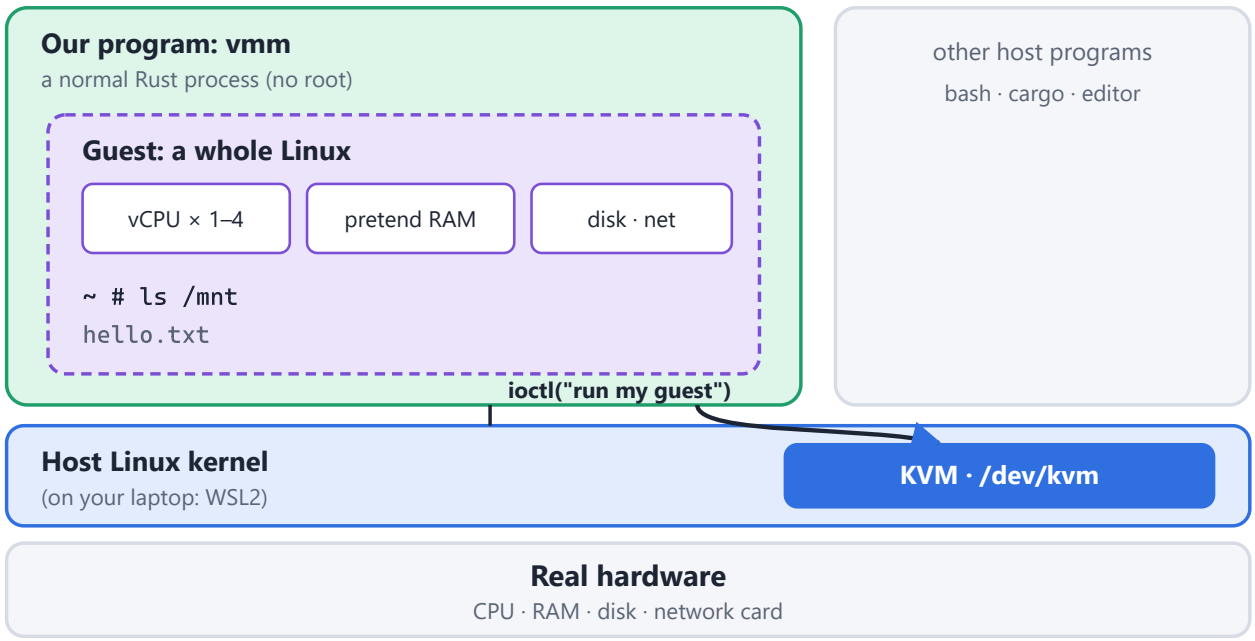
8 Map of the code

9 Project log (we add to this)

1

The project in one picture

> We wrote a program called `vmm` that makes a **pretend computer** inside your real computer. A whole Linux runs inside it and thinks it owns a real machine.



KVM does the fast part (running guest code on the real CPU). Our `vmm` does everything else.

Who does what

Part	Job in one line
KVM (in the host kernel)	Runs the guest's instructions directly on the real CPU, and stops when the guest touches something it can't handle.
vmm (our Rust program)	Builds the pretend machine: memory, CPUs, disk, network, serial console. Answers whenever KVM stops.
Guest (Linux 6.12)	An ordinary, unmodified Linux kernel and a BusyBox shell. It never knows it is pretend.

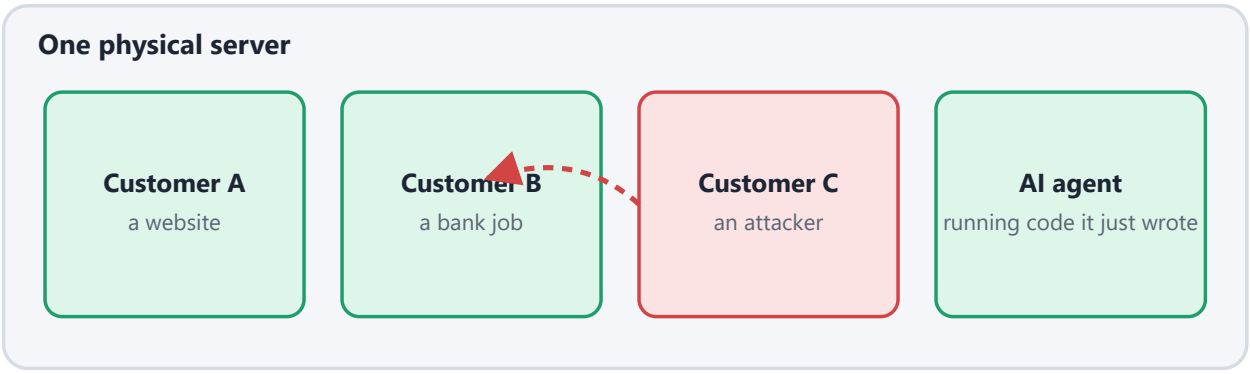
INTERVIEW ANGLE

"VMM" means **Virtual Machine Monitor**. KVM is the *hypervisor* part inside the kernel; the VMM is the user-space program that drives it. QEMU, Firecracker, Cloud Hypervisor and crosvm are all VMMs that sit on top of KVM. Ours is a small Firecracker-style one.

2

The problem: strangers' code on your computer

> A cloud company runs code from **thousands of strangers** on the **same machine**. Any one of them might be an attacker. How do you keep each one in its own room?



The wall between rooms must hold. And rooms must be cheap and fast to build.

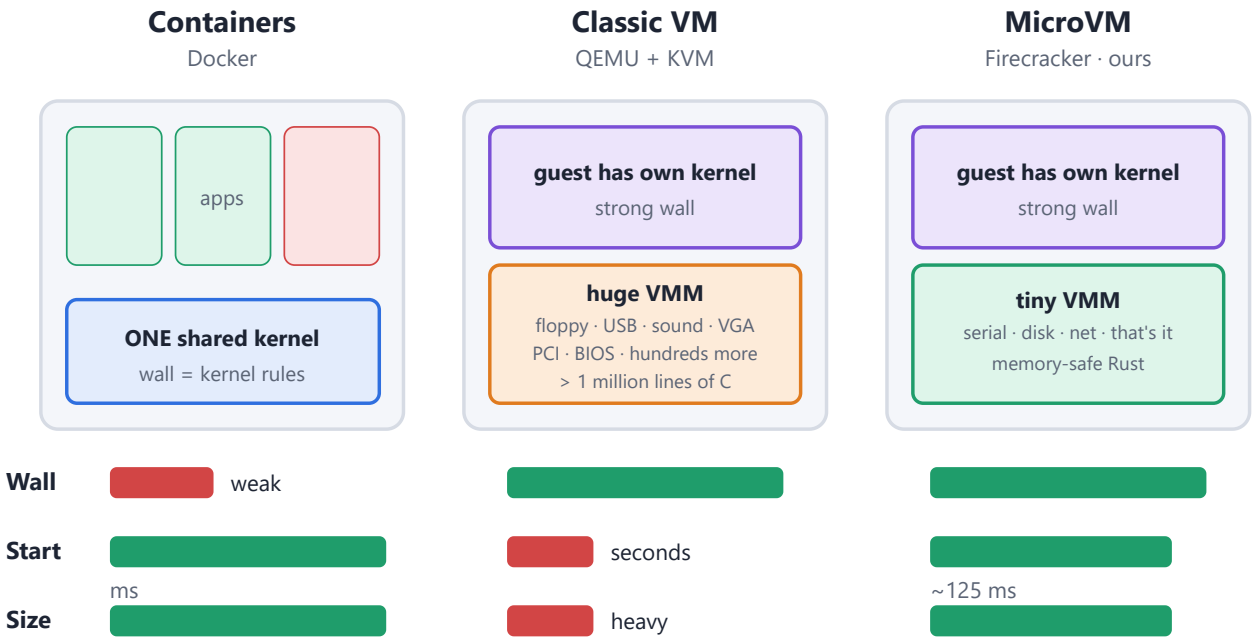
Three things everyone wants at once

Want	Why
Strong walls (isolation)	If C breaks out, it can read A's and B's data. That ends a cloud business.
Fast start	Serverless functions start on demand. Users notice seconds of delay ("cold start").
Tiny size (density)	More rooms per server = less hardware per customer = cheaper.

The old tools gave you **two of the three**. The next page shows why.

3

Three ways to build walls



Longer green bar = better. MicroVM numbers are Firecracker's published figures.

> The trick of a microVM: keep the **strong wall** of a real VM, but throw away every device a cloud server doesn't need. Less code = fewer places to break in, and faster boot.

COMMON MISTAKE

"A container is a small VM." No. A container is a normal process with kernel-enforced limits; every container shares the **same** host kernel. One kernel bug can break every wall at once. A VM gives each tenant its **own** kernel, and the only shared surface is KVM plus the VMM.

4

Who uses this in industry

The same shape as our project (KVM + a small Rust VMM) is running in production at large scale.

Firecracker

Amazon Web Services

Built for AWS Lambda and Fargate. Each function call runs in its own microVM. Open-sourced 2018. The model for our project.

Cloud Hypervisor

Intel, Microsoft, others · Linux Foundation

A Rust VMM for modern cloud workloads. Shares the `rust-vmm` crates we use.

crosvm

Google

Runs Linux apps on ChromeOS and protected VMs on Android. The original Rust VMM that Firecracker grew from.

Kata Containers

OpenInfra

Looks like a container to Kubernetes, but each pod is secretly a microVM. Can use Firecracker or Cloud Hypervisor.

Fly.io and similar hosts

app platforms

Run customers' apps as Firecracker microVMs around the world.

AI code sandboxes

e.g. E2B and agent platforms

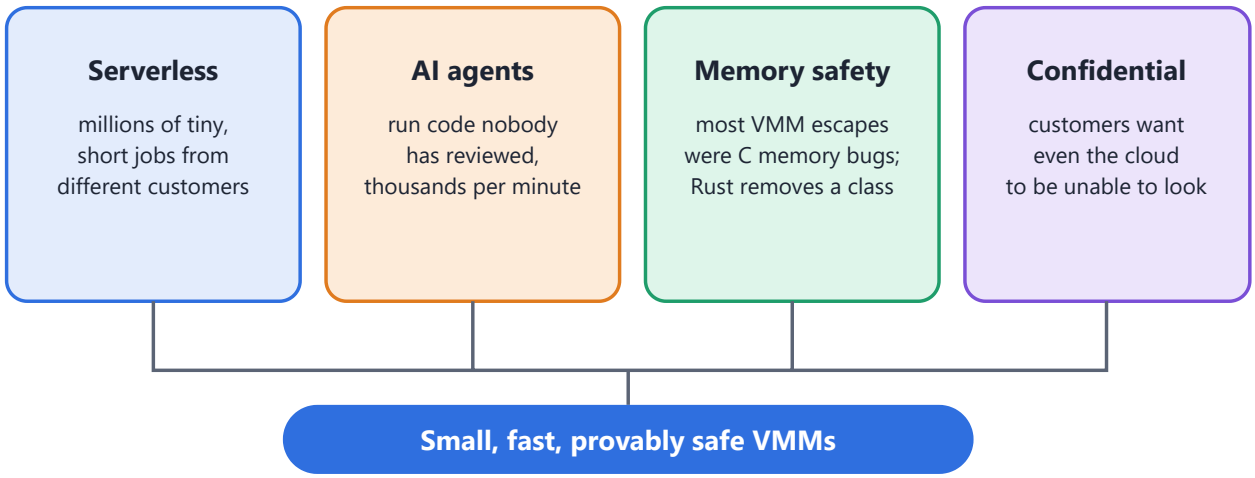
When an AI agent writes code and runs it, that code is untrusted. Many sandboxes put each run in a microVM.

IN THIS REPO

We build on the same **rust-vmm** crates these projects share: `kvm-ioctls`, `vm-memory`, `linux-loader`, `vm-superio`, `seccompiler`, `event-manager`. Learning this repo is learning their building blocks.

5

Why now, and what they are chasing

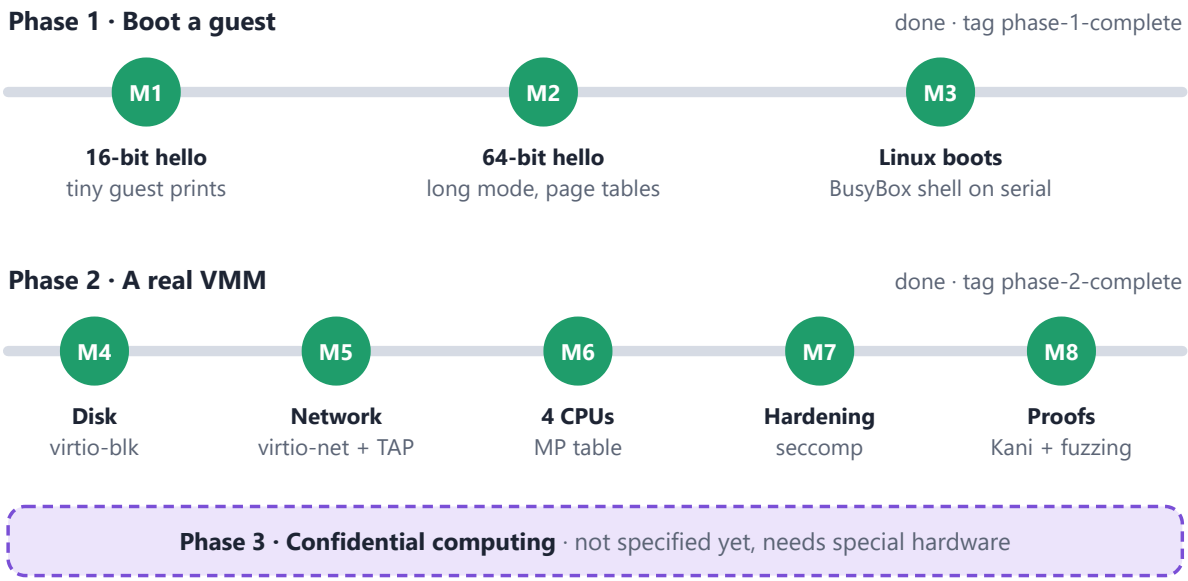


What the industry is trying to achieve

Goal	How it shows up in our project
Smallest possible attack surface	Only serial, i8042 (for reboot), virtio-blk, virtio-net. No PCI, no ACPI, no BIOS.
Boot in a blink	We load the Linux kernel directly into guest RAM and jump in. No firmware step.
Assume the VMM itself might get hacked	M7: seccomp lets each thread make only the system calls it needs. Anything else kills the VMM.
Prove, don't hope	M8: Kani proves the guest-facing parsers can't panic or read out of bounds; a fuzzer throws millions of random inputs at them.
Hide the guest from the host	Phase 3 (not started): AMD SEV-SNP / Intel TDX encrypt guest RAM.

6

Our road: Phase 1 → 2 → 3



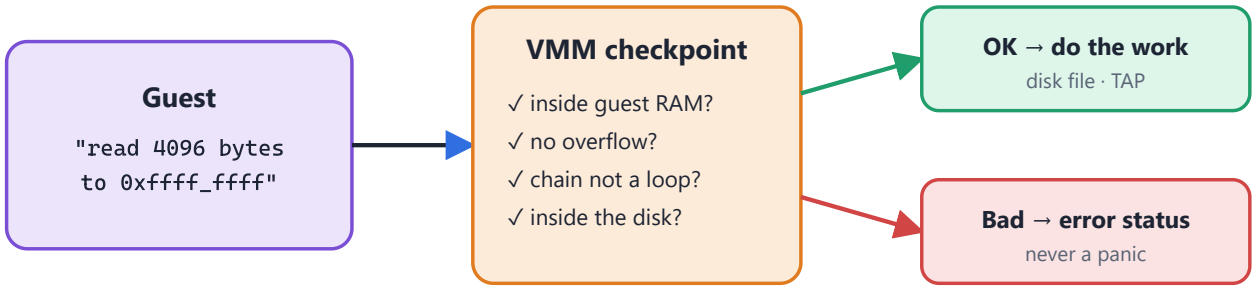
What each step adds, in kid words

Step	Before	After
M1–M2	An empty pretend CPU.	It can run our own tiny program and say hello.
M3	Only tiny programs.	Real Linux runs; you type commands in a shell.
M4	Everything vanishes on reboot.	A disk that remembers (a file on the host).
M5	The guest is alone.	The guest can `ping` the host and download files.
M6	One brain.	Up to four brains working at once.
M7	If the VMM is tricked, it can do anything we can.	Each thread may only do its own job; anything else kills it.
M8	"We think the parser is safe."	A computer checked every possible input (within limits).

7

The one rule: the guest is a stranger

> Every number the guest hands us (an address, a length, an index) might be a lie. We **check it** before we use it. The guest must never be able to crash, hang, or trick the VMM.



Three layers of defence: checks in code (M4–M5) · proofs and fuzzing of the checks (M8) · seccomp if the checks still fail (M7)

IN THIS REPO

This rule is written in the project `SPEC.md` : "A guest is untrusted: nothing a guest does (any port write, any MMIO access, any byte value) may panic the VMM."

8

Map of the code

About 6,000 lines of Rust. Here is where each idea lives.

File / folder	What it is
`src/main.rs`	Command line: `vmm raw` (tiny guests) and `vmm boot` (Linux).
`src/lib.rs`	Puts the machine together and starts it.
`src/vm.rs`	Opens `/dev/kvm`, creates the VM, adds guest RAM, interrupt controller.
`src/vcpu.rs`	One thread per pretend CPU; the `KVM_RUN` loop and its exits.
`src/loader.rs`	Copies the Linux kernel and initramfs into guest RAM.
`src/arch/x86_64/`	x86 setup: registers, GDT, page tables, boot params, MP table.
`src/devices/`	Serial port, i8042 (reboot), port and MMIO buses, TAP.
`src/devices/virtio/`	Virtqueues, the MMIO transport, block and net devices, the checked parsers, Kani proofs.
`src/event_loop.rs`	Main thread: waits for disk/net doorbells and does the I/O.
`src/seccomp.rs`	Per-thread allowlists of system calls.
`src/terminal.rs`	Puts your terminal in raw mode and always restores it.
`fuzz/`	Separate crate with the fuzz target.
`guests/ scripts/ tests/`	Tiny assembly guests, build/verify scripts, integration tests.
`specs/ notes/`	The specs (what to build), decisions log, learning notes per phase.

CHECK YOURSELF

1. In one sentence, what is the difference between KVM and our `vmm` ?
2. Why is a container's wall weaker than a VM's wall?
3. Name the three things a cloud provider wants at once, and which old tool misses which.
4. Why does throwing away devices (floppy, USB, PCI) make the VMM safer, not just smaller?
5. Our three layers of defence against a hostile guest: what are they?

9

Project log

Things we add about the project over time, newest at the bottom.

> 2026-10-02
>
> File started. Covers what the project is, the problem, where industry uses microVMs, and the Phase 1 → 3 road.